

Grupo 11:

Bustos Miguel Angel – 39728123 – P.U

Sueldo Mariano Leonel – 39976244 – P.U y L.I

Orlando Mauro Valentin – 42797700 – P.U

Valle Guillermo Augusto – 39481225 – P.U y L.I

Link del Repositorio: <https://github.com/sueldoM/tp-pruebas-software-grupo-11.git>

TP1 – Respuestas conceptuales

Lenguaje y framework: C# (.NET 10), MSTest 4 y Moq (para los dobles de prueba).

1. Configuración inicial y pruebas básicas

¿Podemos identificar pruebas unitarias y de integración en el trabajo realizado?

Sí, totalmente. Ambas están presentes y conviven en el proyecto:

- **Pruebas unitarias:** Son aquellas en las que evaluamos un componente en absoluto aislamiento. Por ejemplo, en `ProductoTests` probamos el constructor o el método `ActualizarPrecio` de la clase `Producto` sin involucrar nada más. Lo mismo pasa cuando probamos `AplicarDescuento` en la clase `Tienda` usando un *mock* de `Producto`: ahí le quitamos a la tienda su dependencia real para evaluar únicamente su lógica interna.
- **Pruebas de integración:** Aparecen cuando ponemos a colaborar a dos o más clases reales. Por ejemplo, cuando en `Tienda` agregamos, buscamos o eliminamos un `Producto` real, estamos verificando que ambas piezas ensamblen bien. Lo mismo ocurre en pruebas más complejas como `CalcularTotalCarrito` o los escenarios de `FlujoCompleto_*`, donde encadenamos varios pasos reales (agregar, buscar, descontar y calcular totales).

En la práctica cotidiana, la línea de separación no siempre es tan tajante. Muchos desarrolladores consideran que un test de `Tienda` usando un `Producto` real es un **test unitario sociable** (porque `Producto` es solo un objeto de dominio liviano), mientras que otros lo catalogan directamente como una prueba de integración a pequeña escala.

2. Pruebas con excepciones

¿Habría sido posible escribir las pruebas antes del código? ¿Cómo sería ese proceso?

Sí, de hecho es la esencia de **TDD (Test-Driven Development)**. Si hubiéramos aplicado TDD desde el principio, el flujo de trabajo habría avanzado en ciclos cortos y continuos de tres pasos:

1. **Rojo (Red):** Primero escribimos un test para el nuevo comportamiento que queremos agregar (por ejemplo, verificar que al llamar a `EliminarProducto` con un ID que no existe se lance una excepción `KeyNotFoundException`). Ejecutamos la prueba y falla (o ni siquiera compila), lo cual es correcto porque la funcionalidad aún no está implementada.
2. **Verde (Green):** Escribimos el código más simple y directo posible para lograr que la prueba pase. No buscamos elegancia todavía, solo que el test quede en verde.
3. **Refactor:** Limpiamos y optimizamos el código implementado (por ejemplo, haciendo que `EliminarProducto` reutilice la lógica de `BuscarProducto`). Volvemos a correr la suite de pruebas para asegurarnos de no haber roto nada en el camino.

El gran beneficio de este enfoque es que nos obliga a definir los requisitos mediante código ejecutable antes de programar la solución, evitando desarrollar características innecesarias o lógica "por si acaso".

3. Dobles de prueba

¿Dónde encontramos los "Controladores" y "Resguardos"?

- **Controlador (*driver*):** Es el componente encargado de invocar e impulsar la ejecución de la unidad bajo prueba. En nuestro proyecto, ese rol lo cumplen los propios métodos anotados como tests dentro del entorno de MSTest, los cuales ejecutan los métodos de `Tienda` o `Producto` y evalúan sus resultados.
- **Resguardo (*stub*):** Es un componente simplificado que suplanta a una dependencia real para proveer respuestas prefijadas. Un claro ejemplo es el `Mock<Producto>` que usamos al probar `AplicarDescuento`: no ejecuta lógica real de un producto, sino que simplemente retorna un precio fijo (por ejemplo, `Precio = 200`) y nos permite registrar cómo reacciona la tienda.

¿Qué es exactamente un "test double" y qué variantes existen?

El término **test double** (doble de prueba) se inspira en los dobles de riesgo del cine: es un objeto o función sustituta que reemplaza a una dependencia real durante una prueba para aislarla, evitar operaciones lentas/impredecibles (como bases de datos o red) y tener control total del escenario.

Aunque en la diaria solemos llamarle "mock" a todo, en ingeniería de software se clasifican según su propósito:

- **Dummy:** Objetos "de relleno" que se pasan como argumento solo para satisfacer la firma de un método, pero nunca se leen ni se usan.
- **Stub:** Devuelven respuestas o valores fijos ante llamadas específicas para que el test avance por un camino determinado.
- **Spy:** Actúan como un *stub*, pero además van registrando internamente de qué manera fueron invocados (cuántas veces, con qué parámetros, etc.).
- **Mock:** Tienen expectativas preprogramadas sobre qué métodos *deberían* ser llamados durante el test. Si esas llamadas no suceden tal como se configuraron, la prueba falla (por ejemplo, mediante un `.Verify()`).
- **Fake:** Tienen una implementación funcional real, pero simplificada para pruebas (como guardar datos en una lista en memoria en lugar de una base SQL real).

4. Fixtures

¿Qué es un fixture?

Un *fixture* es el entorno o estado conocido que configuramos de manera previa a la ejecución de los tests. Nos garantiza que cada prueba comience siempre bajo las mismas condiciones exactas. En nuestro caso, el método `Setup` prepara una instancia de Tienda cargada con un conjunto predeterminado de productos (Remera, Pantalón, Zapatillas).

Ventajas y enfoque de diseño de pruebas

Utilizar un *fixture* nos aporta varias ventajas clave:

- **Mantenibilidad:** Elimina la duplicación de código de inicialización en cada método de prueba.
- **Aislamiento y repetibilidad:** Al reiniciarse antes de cada test, evita que el resultado de un test afecte o contamine a los demás.
- **Claridad:** El código del test queda enfocado únicamente en la acción y la verificación que le interesan.

En cuanto al enfoque, las pruebas son predominantemente de **caja negra**, ya que interactúan con la interfaz pública de la aplicación analizando entradas y

salidas sin preocuparse por la implementación interna. Sin embargo, hay pinceladas de **caja blanca** cuando usamos *mocks* para verificar invocaciones internas específicas (como confirmar la llamada a ActualizarPrecio). El *fixture* en sí no define la estrategia, simplemente actúa como el andamiaje de preparación del entorno.

Setup vs. Teardown

- **Setup** ([TestInitialize] en MSTest): Es el bloque que se ejecuta automáticamente *antes* de cada test para construir el escenario inicial (crear la tienda y poblar el inventario).
- **Teardown** ([TestCleanup] en MSTest): Es el bloque que se ejecuta *después* de cada test para limpiar los recursos utilizados (vaciar colecciones, cerrar conexiones o eliminar archivos). Garantiza que el entorno quede limpio incluso si la prueba falla.

5. Integración y cobertura

Procedimiento para la recolección y generación del reporte de cobertura

Para medir la cobertura del código se utilizó la herramienta Coverlet integrada con .NET, sumada a ReportGenerator para la transformación del archivo raw XML a un informe web gráfico e interactivo.

Pasos ejecutados en la terminal:

- **Instalación del recolector en el proyecto de pruebas:**

```
dotnet add TiendaApp.Tests package coverlet.collector
```

- **Ejecución de tests con recolección de cobertura:**

```
dotnet test --collect:"XPlat Code Coverage"
```

Esto generó un archivo coverage.cobertura.xml dentro del directorio TiendaApp.Tests/TestResults/<GUID>/.

- **Instalación de la herramienta global de generación de reportes:**

```
dotnet tool install -g dotnet-reportgenerator-globaltool
```

- **Procesamiento del informe en HTML:**

```
reportgenerator "-reports:**/coverage.cobertura.xml" "-targetdir:coveragereport" -reporttypes:Html
```

- **Visualización:**

Se abrió el archivo [coveragereport/index.html](#) en el navegador web para consultar el análisis detallado.

Análisis de métricas y tipos de cobertura utilizados

A partir del informe HTML generado, se corroboraron las siguientes métricas sobre el proyecto TiendaApp:

- **Tipos de cobertura medidos:**
 - **Cobertura de líneas / sentencias (*Line coverage*):** Mide la proporción de instrucciones ejecutadas por la suite de pruebas respecto al total existente.
 - **Cobertura de ramas / decisiones (*Branch coverage*):** Evalúa si las expresiones condicionales (instrucciones if, validaciones de argumentos, rangos de precio o porcentaje) fueron evaluadas en sus distintos caminos posibles (true y false).
- **Resultados obtenidos:** La suite de 19 pruebas unitarias e integradas alcanzó valores elevados de cobertura en los módulos del proyecto, garantizando que tanto los escenarios de éxito (*happy paths*) como el manejo de excepciones (descuentos inválidos, montos negativos, ítems inexistentes) estén contemplados.
- **Limitaciones conceptuales de la métrica:** Es clave remarcar que una cobertura elevada indica **qué líneas de código fueron ejecutadas**, pero no garantiza por sí sola la **calidad de las aseveraciones (*asserts*)**. Una prueba puede cubrir el 100% de las líneas de un método y aun así no verificar adecuadamente el resultado esperado o dejar de lado casos de borde lógicos.

Estrategia de integración ascendente (Bottom-Up)

En una estrategia de **integración ascendente**, el desarrollo y las pruebas comienzan desde los módulos de menor nivel (la base de la arquitectura) y van avanzando gradualmente hacia las capas superiores. Cuando una capa superior aún no ha sido desarrollada, se recurre a un **driver** (controlador) que simula su comportamiento para poder probar las capas inferiores.

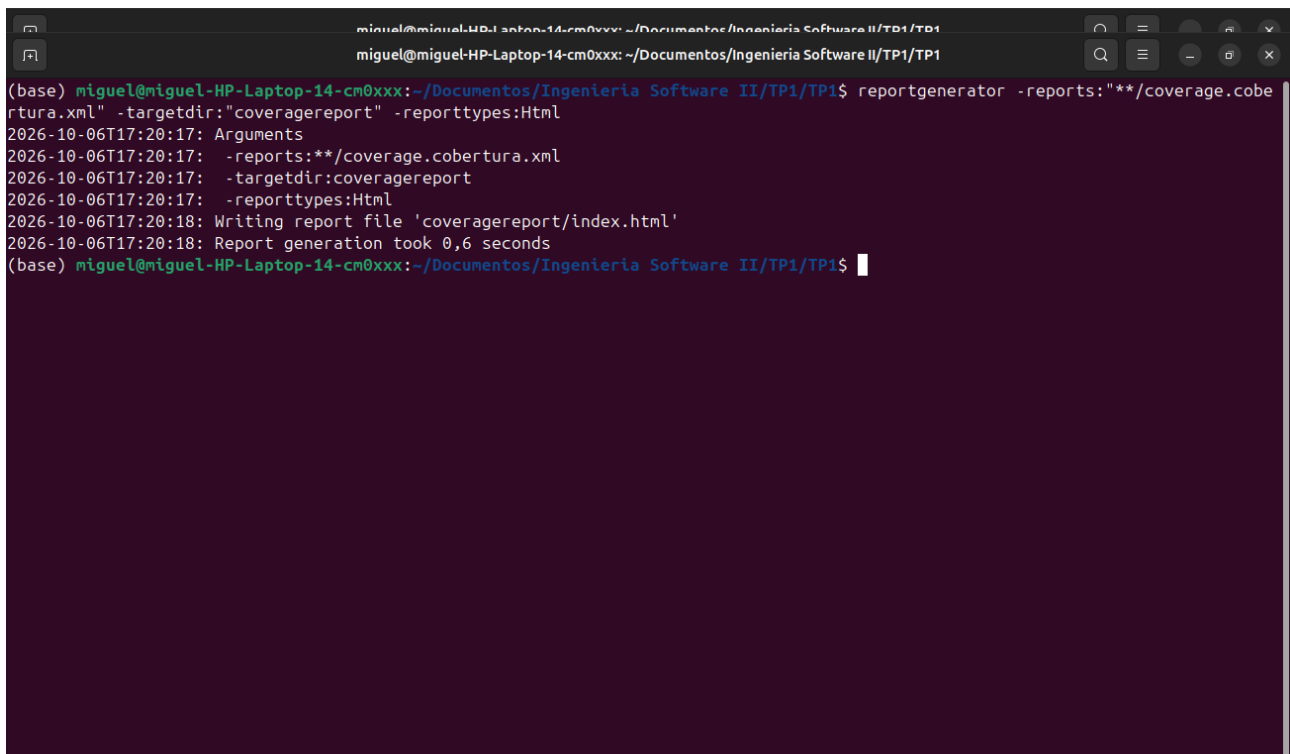
Ejemplo práctico en la tienda:

1. **Nivel inferior:** Primero implementamos y probamos de manera aislada la clase Producto (con su método ActualizarPrecio), ya que no depende de ningún otro módulo.
2. **Nivel intermedio:** Integración de la clase Tienda sobre Producto. Validamos cómo conviven las entidades reales al gestionar el inventario y aplicar descuentos.
3. **Nivel superior:** Supongamos que más adelante desarrollaremos un módulo de CajaRegistradora o CarritoCompras que consumirá los

métodos de Tienda. Mientras ese módulo superior no exista, nuestros tests de integración actúan como el *driver* encargado de invocar a Tienda y verificar su funcionamiento integrado.

De esta forma, la arquitectura se valida paso a paso desde los cimientos hacia la superficie, haciendo mucho más sencillo identificar la causa raíz de cualquier falla.

Imágenes sacadas de la cobertura

A terminal window with a dark background and light-colored text. The window title bar shows the path ~/Documentos/Ingenieria Software II/TP1/TP1. The terminal content shows a command being executed in a base environment, followed by several lines of output including arguments, file paths, and a completion message.

```
(base) miguel@miguel-HP-Laptop-14-cm0xxx:~/Documentos/Ingenieria Software II/TP1/TP1$ reportgenerator -reports:"**/coverage.cobertura.xml" -targetdir:"coveragereport" -reporttypes:Html
2026-10-06T17:20:17: Arguments
2026-10-06T17:20:17:   -reports:**/coverage.cobertura.xml
2026-10-06T17:20:17:   -targetdir:coveragereport
2026-10-06T17:20:17:   -reporttypes:Html
2026-10-06T17:20:18: Writing report file 'coveragereport/index.html'
2026-10-06T17:20:18: Report generation took 0,6 seconds
(base) miguel@miguel-HP-Laptop-14-cm0xxx:~/Documentos/Ingenieria Software II/TP1/TP1$
```

